Food and Humanity 1 (2023) 800–808

Contents lists available at ScienceDirect

Food and Humanity

journal homepage: www.editorialmanager.com/foohum/journal_overview.html

A comparative study of deep learning methods for food classification with images

Geerish Suddul *, Jean Fabrice Laurent Seguin

University of Technology, Mauritius (UTM), Pointe-aux-Sables, Mauritius

ARTICLE INFO

Keywords:
Deep Learning
Convolutional Neural Networks
Computer Vision
Transfer Learning
Food classification
Food-11 Dataset
Food-101 Dataset

ABSTRACT

The monitoring of food intake plays a key role in avoiding different health problems such as diseases, but also difficulties linked with mental and physical wellbeing. Food monitoring consists firstly of, identifying food types and secondly, recommending the appropriate food for better nutrition. This study addresses the aspect of identifying and classifying food types using Artificial Intelligence (AI) through computer vision, deep learning and transfer learning techniques. We used a dataset containing over 16000 food images, classified into 11 categories: Bread, Dairy product, Dessert, Egg, Fried food, Meat, Noodles/Pasta, Rice, Seafood, Soup, and Vegetable/Fruit. On average, an additional 15884 images were computationally generated for each experiment with data augmentation techniques such as rotation, shifting, shearing, zooming and flipping. Our experiments with different Convolutional Neural Networks (CNN) demonstrate that transfer learning with the EfficientNetV2 model achieved a significant validation accuracy of 94.5 % in classifying the different types of food from images. Also, data augmentation contributes to deal with the imbalanced dataset, with the model achieving an F1-score of 94.7 %. Together with drop out and early stopping techniques, data augmentation also contributes to keep the model safe from overfitting.

1. Introduction

Economic development has undeniably brought along changes in human lifestyles. There is an increase in the standard of living with better educational systems, more jobs and higher income levels (Zakour et al., 2018). However, besides beneficial changes, the shift in nutritional patterns is raising several concerns. Foods rich in fats, sugar and salt are being favoured over fruits, vegetables and other dietary fibres, resulting in an increase in noncommunicable diseases (NCDs) (WHO, 2020). The consumption of a healthy diet, therefore, is gaining more importance with the rise of diseases like diabetes, stroke and cancer among others.

In order to address the nutrition problem in food consumption, it is important to firstly, identify which type of food is being consumed, and consequently recommend changes to improve health conditions. The evolution of machine learning, which is a subset of Artificial Intelligence, into deep learning has created unprecedented advances in various scientific domains. Especially, with the advent of computer vision techniques for appropriate identification of contents from images and videos. The problem of food identification and classification from images is still a challenge given its complex composition of patterns (Kiourt, Pavlidis, & Markantonatou, 2020; Şengür, Akbulut, & Budak, 2019; Vijayakumari et al., 2022). An image for the same type of food varies in terms of contents, shapes, size, textures and colours based on either availability of ingredients, or location or cultural traditions. This problem requires advanced deep learning techniques which can learn to identify specific characteristics of different types of dishes. Nevertheless, applying deep learning techniques to identify the features which defines an image may overfit (Xu et al., 2019a. 2019b; Qian et al., 2020a, 2020b; Kim et al., 2021). This usually happens when the model learns too much details from a limited set of training images, which results in a high risk of underperforming while classifying new unseen images. For example, the image of a dessert, during training, may contain an object such as a spoon or a fork while the image of the same dessert, used for prediction, may not contain any additional objects.

Convolutional Neural Networks (CNNs) are relatively recent computer vision techniques based on deep learning, which provides better performance in image recognition. Various studies have applied this technique for automatic food type recognition, achieving different levels of accuracy. Using the Food-101 dataset, Bossard et al. (2014)

* Corresponding author.
E-mail address: g.suddul@utm.ac.mu (G. Suddul).

https://doi.org/10.1016/j.foohum.2023.07.018
Received 29 March 2023; Received in revised form 5 July 2023; Accepted 18 July 2023
Available online 1 August 2023

configured a deep CNN model from scratch based on the AlexNet approach (Krizhevsky et al., 2012). Their work outperformed all other food recognition techniques, by achieving an average accuracy of around 51 % by mining discriminative components using Random Forests algorithm. With different CNN configurations, Kagaya et al. (2014) further improved the prediction accuracy to around 74 % for 10 classes of dishes. Most of these approaches are time consuming, requiring tedious parameter configuration for the models. With transfer learning, pre-trained models with minimum configurations have proved to be less time consuming and more effective (Kawano & Yanai, 2014; Yanai & Kawano, 2015; Attokaren et al., 2017). The work of Islam et al. (2018a, 2018b) on the Food-11 dataset, demonstrates that their pre-trained InceptionV3 model (Szegedy et al., 2016) achieved an accuracy of around 93 % while their CNN built from scratch only achieved around 75 %. Improvements on CNN models have also evolved and more recently, the EfficientNet (Tan & Le, 2021a, 2021b) approach demonstrated major improvements with better accuracy and lower training times for different computer vision problems. Vijayakumari et al. (2022) applied the EfficienNetB0 to the Food-101 dataset, resulting in an accuracy of 80 % which outperforms all other benchmarked deep learning techniques for food recognition. Different machine learning techniques utilised for the classification of food images have brought along significant improvement in the accuracy of prediction over the years. However, the field of computer vision has further evolved, with increased computer resources, larger datasets and improved Convolutional Neural Networks (Foret et al., 2021; Tan & Le, 2021a, 2021b). These enhancements can contribute to further increase the accuracy of food classification from images. Also, in this study we omitted previous results which claim to have achieved very high, near perfect accuracy without justifying that their approaches are not overfitting.

Dhanya et al. (2022) explores the application of computer vision technologies based on deep learning that can assist in the different phases of agriculture, from preparation to harvesting. Modern approaches such as Generative Adversarial Networks (GAN) and Vision Transformers (ViT) have been considered. Their findings indicate that CNN are the foundation of modern computer vision techniques and the success of contemporary approaches rely on quality dataset. The work of Lopes et al. (2022) compares a Deep Computer Vision System (DCVS) and a traditional Computer Vision System (CVS) for classifying cocoa beans into different varieties. The DCVS, using Resnet18 and Resnet50 as backbone models, achieved the highest accuracy of 96.82 %, while the CVS using Support Vector Machine (SVM) achieved an accuracy of 85.71 %. The study also discusses the influence of handcrafted features on cocoa bean classification and applies Class Activation Maps to visualise the most important regions of the images in the DCVS model. Oliveria et al. (2021) used computer vision to extract hand-crafted features from the cocoa beans, which were then used as predictors in random decision forests for classification. The method achieved an accuracy of 93 % for an unbalanced dataset and 92 % for a balanced dataset, demonstrating its potential as a replacement for the traditional cut-test method in classifying fermented cocoa beans. More recently, the application of Gradient-weighted Class Activation Mapping (Grad-CAM) (Selvaraju et al., 2020) to visualise learned features from images, opened a new avenue for the assessment of food images classification models. This technique visually marks every class identified in an image, predicting every pixel that belongs to each class, usually represented with a heatmap. Shao et al. (2023) applied a multimodal feature fusion together with a multiscale fusion to build an RGB-D fusion network to enhance the identification of nutrients from food images. Their approach assessed with Grad-CAM demonstrates low error rates of 15.0 % for calorie and 10.8 % for mass detection. The effectiveness of the Grad-CAM approach has also been demonstrated for the identification of the most significant features for the separation of ingredients in the work Ma et al. (2022) and Zhu et al. (2021). While nutrient and volume identification remains a challenge for computer vision with machine learning, Grad-CAM assessment technique helps to determine where improvements are to be made.

This work focuses mainly on exploring the most efficient computer vision technique based on machine learning for food detection from images. The aim of this study is to improve the automatic identification and classification of food types from images. The objectives are firstly to identify an appropriate dataset of food images. Secondly to assess the effectiveness of deep learning models built from scratch and thirdly to use transfer learning by applying models with pre-learned parameters. While different models have been applied in this context, this study focuses on determining an approach which generalizes well on unseen images by avoiding overfitting.

2. Background

This section presents the fundamental deep learning concepts so as to establish a basic level of understanding.

2.1. Convolutional neural network

A Deep Neural Network (DNN) has to deal with an excessively large number of parameters to learn the features from an image, making the process computationally intensive. Food images are usually coloured and thus consists of three different channels, Red, Green and Blue (RGB). An image having a height and width of 1000 pixels each, results in (1000 x 1000 x 3) 3000000 input parameters. When these are fed into a neural network having 1000 neurons in its first layer, a massive 3 billion, computationally demanding, parameters will be generated for processing. The solution to this problem resides with the Convolutional Neural Network (CNN) approach, which significantly reduces the number of parameters through the use of reusable filters, also known as kernels, to detect multiple features in an image.

As can be depicted in Fig. 1, a Convolutional Neural Network (CNN) works with images at pixel level in a matrix form, with three different channels for the Red, Blue and Green (RGB) values. In this example, it is assumed that the input image has a height of 4 pixels, a width of 4 pixels and 3 channels. The CNN is mainly divided into a set of convolutions and pooling layers which extract features from the input image and combine them into a feature map which is flattened into a one-dimensional matrix. This matrix becomes the input to a set of fully connected layers as with a traditional Deep Neural Network (DNN) model. The output of the fully connected layer is usually calculated using a softmax function when there are more than one possible class. In this study, we are working with 11 different classes: Bread, Dairy product, Dessert, Egg, Fried food, Meat, Noodles/Pasta, Rice, Seafood, Soup, and Vegetable/Fruit.

2.1.1. Convolutions

Convolutions adapt very well to inputs which take the form of matrices. From Fig. 2, an input image is represented as a (4 x 4 x 1) input matrix corresponding to pixel values in only one channel. A (2 x 2 x 1) kernel, also known as a filter, is used for convolution, with stride set to 1 and padding to 0. The number of channels for the filter must be the same as for the input. The convolution consists of applying the filter on the input resulting in an activation map (output). Starting at the upper left of the input matrix, the first 4 elements as highlighted, are multiplied with values from the kernel and summed to get the first value of the output matrix, which is 24. With a stride of one, the next 4 elements start at the first row and second column from the input matrix, and the multiplication and summation is repeated to get the second value of the output matrix, which is 22. The operation is repeated, moving the kernel each time by one column vertically at first, and then by one row horizontally to produce the output matrix with 3 rows and 3 columns.

In this particular example, we have used only one filter, hence a single output matrix without any channels. However, using another filter on the same set of input, will create an output with 2 channels, stacked on top of each other. The number of channels for the output

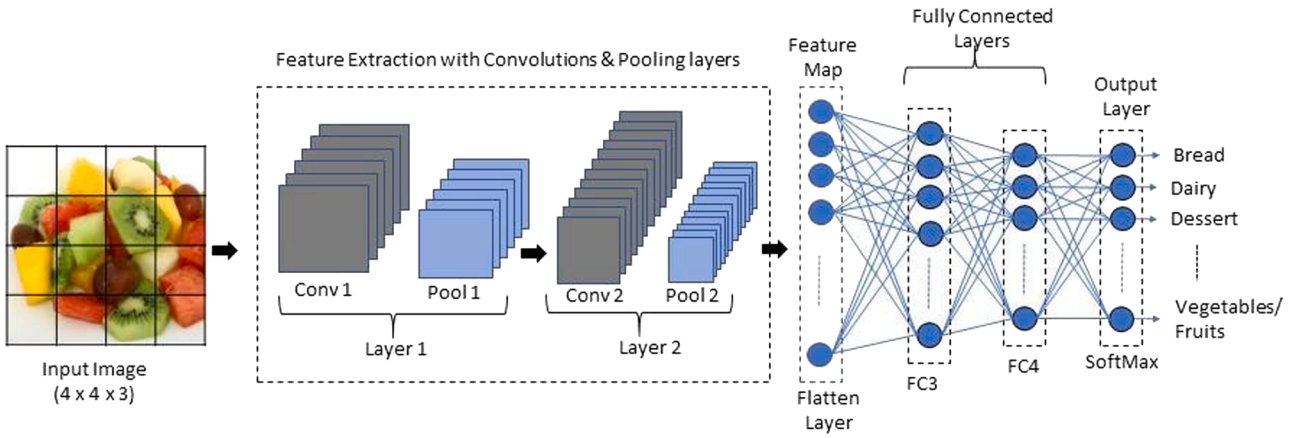

Fig 1. Detailed architecture of convolutional neural network.

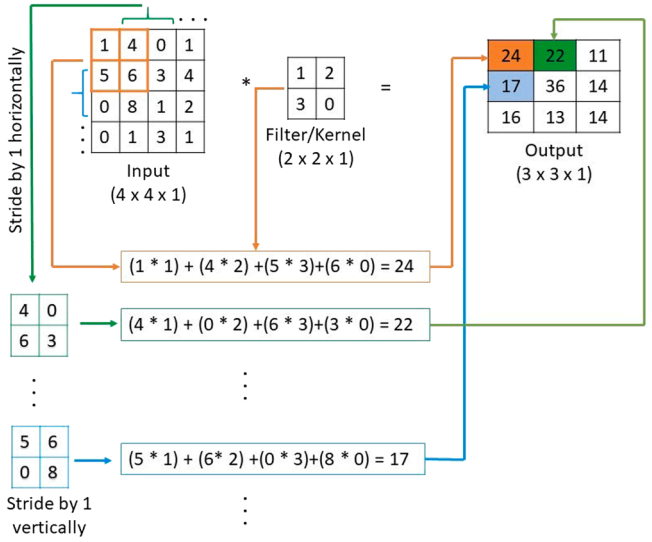

Fig 2. Convolutions Using a single 2x2 Filter.

depends on the number of filters used, hence represented by n_c'. Using stride s and padding p, the size of the output matrix is given by Eqn 1, where $n \times n$ represents the size of the input matrix and $f \times f$ the size of the filter with the same number of channels.

Size of the Input *matrix* : $(n \times n \times n_c)$

Size of the Filter matrix : $(f \times f \times n_c)$

Size of the output matrix using convolutions:

$$\frac{(n-f)+2p}{s}+1 \times \frac{(n-f)+2p}{s}+1 \times n_c' \quad (1)$$

2.1.2. Pooling

The pooling layer basically compresses the size of the output from a convolution, decreasing the number of parameters for computation. There exist different pooling approaches such as max pooling, average pooling, weighted average pooling, L2 norm among others (Bengio et al., 2017). In Fig. 3, max pooling is applied to a (4 x 4 x 1) matrix, using a (2 x 2 x 1) filter and a stride of 2. The output is a (2 x 2 x 1) object, which is almost invariant to small translations of the input. It helps to identify if a feature is present, rather than where it is exactly located in the image.

The size of the pooling output matrix is determined by applying the function from Eqn 2.

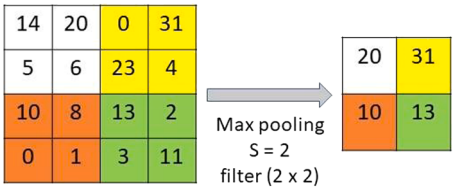

Fig 3. Max Pooling example with 2x2 filter and stride of 2.

$$\frac{(n-f)}{s}+1 \times \frac{(n-f)}{s}+1 \times n_c' \quad (2)$$

2.1.3. Detector stage

The convolution is the first stage or a CNN which produces a set of linear parameters from the input image. This linear representation cannot be used to represent images which are non-linear. Therefore, before applying pooling layers, the output of the convolution is converted from linear to non-linear with the application of the Rectified Linear Unit (ReLU) function. This is usually known as the Detector Stage.

2.1.4. Existing CNN configurations

Building a CNN from scratch can be quite complex and time consuming. An initial set of convolution and pooling layers must be defined along with the fully connected layers, output and the whole set of hyperparameters (stride, padding, pooling strategy, learning rate, batch size, etc) for the model. An experiment will then determine the effectiveness of the model. If it is unsatisfactory, the whole process is repeated again until the model is effective enough.

Another approach is to use configurations of those models which have proven to be effective in different computer vision problems, as mentioned in Table 1.

2.2. Transfer Learning

Transfer learning usually takes an existing CNN model which has been trained and tested on a large dataset of images from different contexts. The pretrained model already has its set of parameters, also known as weights, for each and every layer. Therefore, computational resources are not wasted in finding these parameters again. These pre-trained models are sometimes the result of long training hours with demanding computer resources, on very large openly available datasets. Some of the commonly used open datasets, devised mainly for computer vision are listed in Table 2.

The pre-trained model can be quickly used to train and detect images different from their training context. This transfer of knowledge, based

Table 1
Existing CNN Configurations.

Reference	Model Name	Layers	Number of parameters (approx)
Lecun et al. (1998)	LeNet-5	7 layers: 4 Conv & Pool + 2 FC + 1 Output	60,000
Krizhevsky et al. (2012)	AlexNet	8 layers: 5 Conv & Pool + 2 FC + 1 Output	60,000,000
Simonyan & Zisserman (2015)	VGG	VGG-16: 16 layers	138,000,000
He et al. (2016)	ResNets (Residual Networks)	ResNet-152: 152 Layers	60,000,000
Szegedy et al. (2016)	Inception Network	GoogLeNet: 27 layers (22 parametrised)	4000,000
Tan and Le (2021a)	EfficientNet	EfficientNet-B0: 237 layers (grouped in 5 modules)	5300,000

Table 2
Large datasets commonly used to train CNNs for Transfer Learning.

Dataset	Description	Example of Image Classes	Reference
ImageNet	14,197,122 images (21,847 synsets).	Chimpanzee, abacus, basketball, cradle, forklift, necklace, race care, reel, torch, etc	Deng et al. (2009)
ImageNet Large Scale Visual Recognition Challenge (ILSVRC)	Commonly used subset of ImageNet: Contains 1000 object classes out of 1281,167 training images with 50,000 validation images and 100,000 test images		Russakovsky et al. (2015)
Microsoft COCO (Common Objects in Context)	Contains photos of 91 objects types. With a total of 2.5 million labelled instances in 328,000 images.	Dog, umbrella, fork, orange, door, blender, chair, etc	Lin et al. (2015)
PASCAL-VOC 2012	Contains 20 classes. The train/val data has 11,530 images containing 27,450 ROI annotated objects and 6929 segmentations.	Bird, cat, cow, aeroplane, bicycle, boat, dining table, etc.	Everingham et al. (2015)
SUN	131, 072 images with 908 scene categories, 249, 522 Segmented objects & 3819 Object categories	Igloo, badminton court, lighthouse, flood, shed, etc.	Xiao et al. (2010)
Caltech256	256 object categories containing a total of 30607 images.	Bat, grasshopper, backpack, cake, fern, etc.	Griffin et al. (2022)

on a common set of low-level features, is efficient and allows models from one context to achieve good results in another. After a number of convolution and pooling layers, it has been noted that the generated low-level features, such as edges, shapes, geometric changes and changes in lighting, are common in different visual representations (Kiourt, Pavlidis, & Markantonatou, 2020). There are different strategies to use a pre-trained CNN on a different dataset such as keeping all layers unchanged or frozen, and retrain only the output layer. We can also remove and add our own fully connected layers along with the output layer.

3. Materials and methods

This study is based on an empirical approach. As can be depicted in

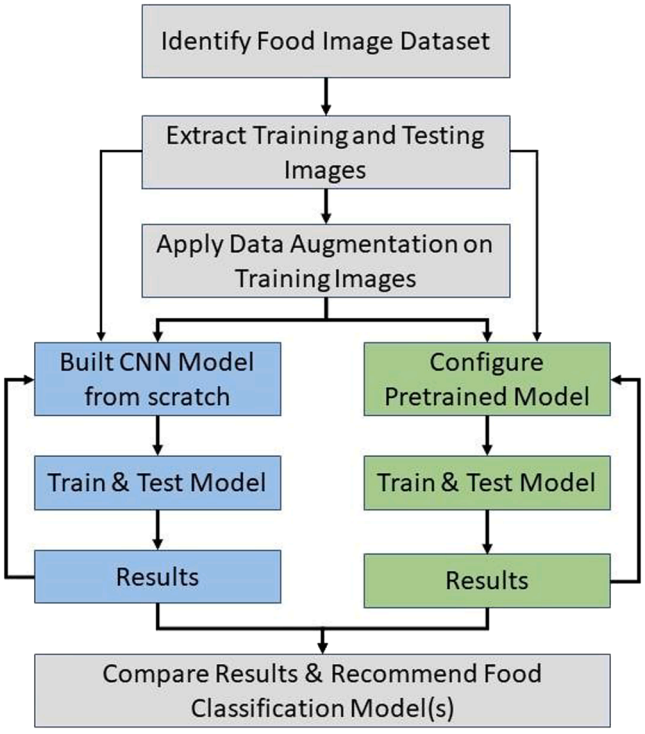

Fig 4. Empirical approach for this study.

Fig. 4, we start by identifying the most appropriate dataset containing food images. The images from the dataset are divided into training and testing sets, to which we apply data augmentation to the training set. This approach helps to further increase the different types of images and reduce overfitting from deep learning models. Following this step, we define two main strategies for our experiments. Firstly, to build and configure a CNN from scratch, defining all the different layers and setting all hyperparameters. Secondly, to identify and use a pretrained model, focusing on the transfer of knowledge gained in a different domain. Both approaches are trained and tested with data augmention and without. If the results are not satisfactory, the configuration of the models are adjusted followed by new experiments. Once the results are conclusive enough, that is the models and their configurations are producing accurate classifications, we present the comparison and recommendations.

3.1. Food image datasets

Over the past decade, a few public food image datasets have become available. Following our literature review, the significant datasets have been shortlisted in Table 3.

While Food-101 is quite common across different research work involving deep learning, it has a large number of classes, which increases the number of parameters and the processing time and adversely affects the training time. Images for the FOOD-11 dataset have been mainly

Table 3
Public food image datasets.

Dataset	Number of Classes	Number of Images
FOOD-5K (Singla et al., 2016)	2	5000
UEC-Food-100 (Matsuda et al., 2012)	100	14461
FOOD-11 (Singla et al., 2016)	11	16643
UEC-Food-256 (Kawano et al., 2014)	256	31651
FOOD-101 (Bossard et al., 2014)	101	101000
Food-524DB (Ciocca et al., 2017)	524	247636
FOOD-101N (Lee et al., 2018)	101	310009

collected from three larger datasets, namely: UEC-FOOD-100, UEC-FOOD-256 and FOOD-101 datasets. Images from social media websites have also been added to those classes with a low number of images. The Food-11 dataset has 16643 images classified into 11 main food classes, in line with the United States Department of Agriculture (Singla et al., 2016) . The USDA further refined the classification of food types into 5 categories only: Vegetables, Fruit, Grains, Protein and Dairy (USDA, 2015). The Food-11 dataset is the closest to these five groups. The different classes of Food-11, with examples are depicted in Fig. 5: (a) Bread, (b) Dairy products, (c) Dessert, (d) Eggs, (e) Fried foods, (f) Meat, (g) Noodles/Pasta, (h) Rice, (i) Seafood, (j) Soup and (k) Vegetable/Fruit.

For each type of food, there is an average of 897 images for training out of a total of 9866 images. An average of 616 images for validation, for each class, out of a total of 6777 images. Fig. 6 shows that the dataset has a relatively fair distribution of images for most classes, except for dairy products, noodle/pasta and rice. To prepare for the experiments, the images were rearranged into training and validation folders where each folder had specific subdirectories for each class, named after their respective classes. The dataset was then loaded into our python program which resized all images to 244 by 244 pixels before converting them into binary format.

In order to increase the variability as well as the number of the images, data augmentation has been performed programmatically. It automatically creates a new set of images in the computer memory, without compromising the existing ones. This further increases the size of the dataset and has the potential to reduce overfitting and thus improving the prediction accuracy. The following approach was adopted for data augmentation:

- Randomly rotate the image within a 0–40 degree.
- Randomly shift the image horizontally along the width (from left to right) using the range 0–0.2.
- Randomly shift the image vertically along the height using the range 0–0.2.
- Randomly shear images with a range of 0–0.2.
- Randomly zoom images with a range of 0–0.2.
- Randomly flip the images horizontally.

The approach used for data augmentation is linked to the number of epochs for training, together with the batch size value and the steps for each epoch. The images are not generated at the same time, but in batches to avoid memory shortage. Each model therefore trains on one batch at a time and for each epoch, the model trains on all the generated images at the end of the training. By introducing drop out and early stopping, images will no longer be generated once the model has converged.

3.2. Deep learning model architecture

3.2.1. Model 1: CNN from scratch

The architecture of this custom-built CNN model consists of 3 convolutional blocks where each block consists of 2 convolution operations with a (3 x 3) kernel with a Rectified Linear Unit (ReLU) activation function and 2 max-pooling operations with a (2 x 2) kernel. Each convolution operation is followed by a max-pooling operation. The number of filters is doubled for each convolutional block. Initially the convolutional block starts with 32 filters, followed by 64 and the last one is 128. After the 3 convolutional blocks, a flatten layer was used to feed the results into a Fully Connected (FC) neural network. This network consists of 2 layers with 512 neurons in the first layer using the ReLU activation function. The last layer of the network has 11 neurons with the activation function set to Softmax to handle multiple outputs. Table 4 sums up the training hyperparameters for Model 1.

The model is set to train for 100 epochs with two call-back functions. The first call-back function stops the training if the accuracy on the validation set reaches 90 % and the other one stops the training if the accuracy on the validation set does not change for 5 epochs, also known as early stopping. Model 1 was trained on both the original and augmented data. Between the last 2 layers, a dropout layer of 0.3 removes neurons with similar weights, to combat overfitting.

3.2.2. Model 2: transfer learning with InceptionV3

The first transfer learning model used for the experiment is the InceptionV3 model. The architecture of the InceptionV3 model is based on a combination of 42 layers. This special CNN introduces an inception module which consists of different convolutions with different kernels.

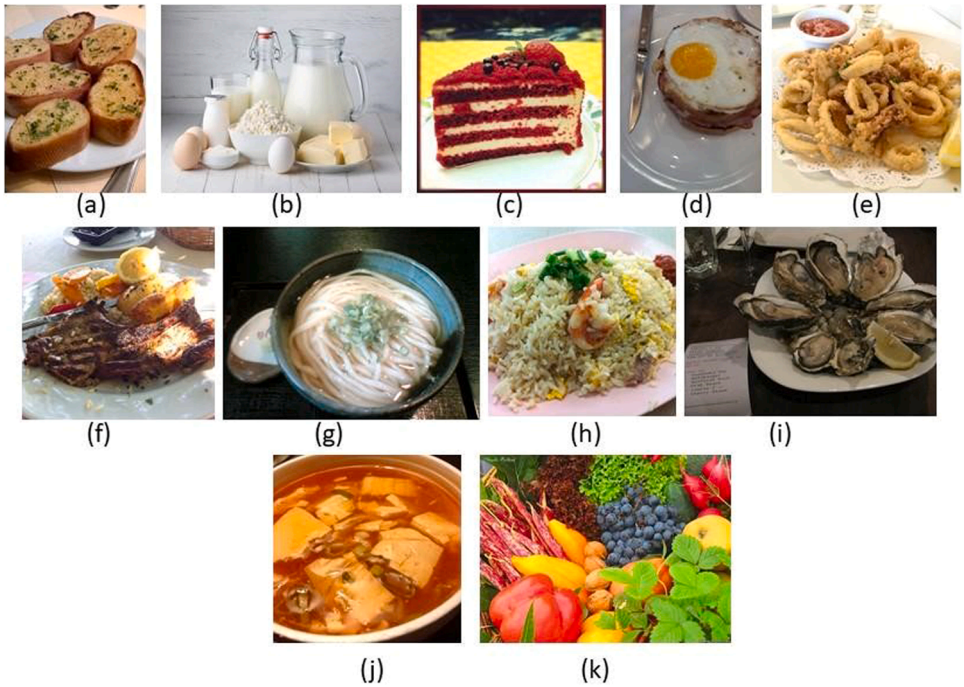

Fig 5. Sample Image for each food class from Food-11.

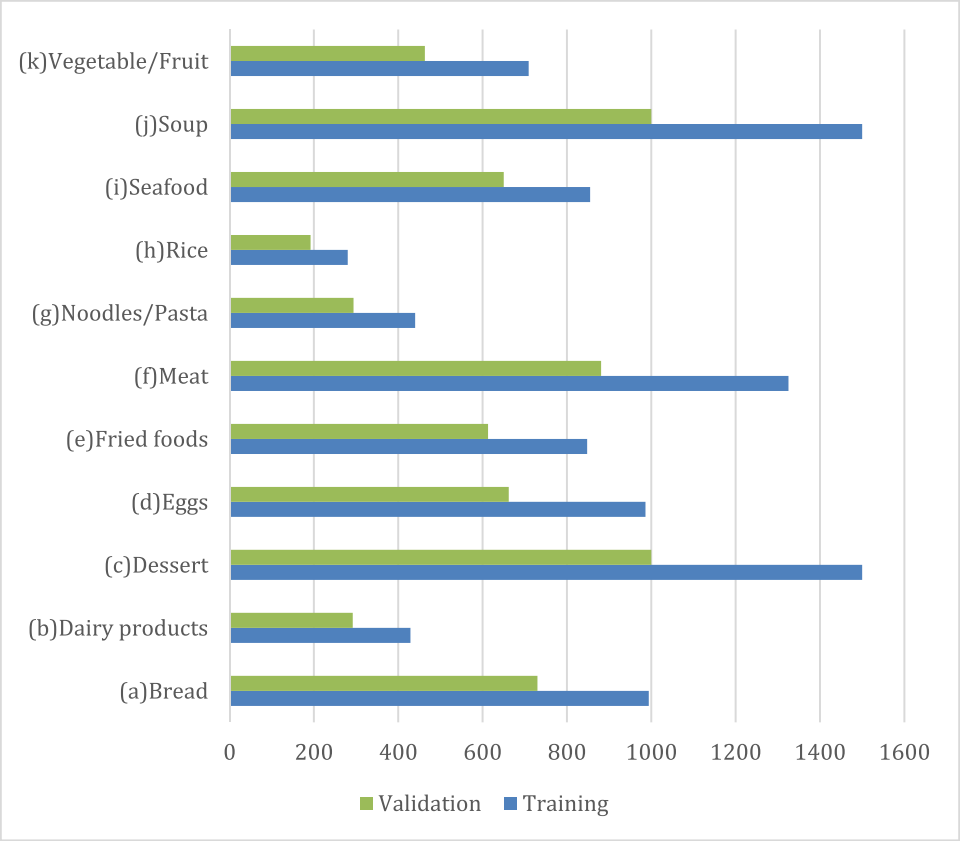

Fig 6. Class distribution for Food-11 dataset.

Table 4
Main hyperparameter configurations for Model 1, Model 2 and Model 3.

Hyperparameter	Value
Loss Function	Sparse Categorical Crossentropy
Optimizer	Adam
Evaluation Metric	Accuracy, Precision, Recall & F1-Score
Max. Epochs	100
Call-back	Accuracy and Early Stopping
Drop-out	0.3 (Model1) /0.5 (Model 2 & Model 3)

The convolution in the inception module consists of 1 x 1, 3 x 3 and 5 x 5 kernels (Szegedy et al., 2016). An InceptionV3 model consists of at least 7 inception modules.

For this study, we used a pre-trained InceptionV3 model available from the Keras Python programming library adapted from Szegedy et al. (2016). The model consists of weights that have been learned from the ImageNet dataset. To train the inceptionV3 model on the food11 dataset, the last convolution layer named Mixed9 has been removed from the pre-trained inceptionV3 model and replaced with a new neural network layer. This layer consists of 2 convolution operations with a kernel of 3 x 3 and 64 filters followed by a batch normalization layer to avoid overfitting. The output is connected to a flattened layer and passed to a dense layer of 512 neurons with the ReLU activation function. A dropout layer of 0.5 has also added followed by a dense layer of 11 neurons with Softmax activation function. The initial training parameters for the InceptionV3 are listed in Table 4.

3.2.3. Model 3: transfer learning with EfficientNetV2

The second pre-trained CNN model is the EfficientNetV2-B2 (Tan & Le, 2021a, 2021b). This solution has been designed to address the problem of scaling in CNN architectures. The uniqueness of the EfficientNet is that it has a special layer called Mobile inverted Bottleneck Convolution (MBConv), which is a convolutional network, based on the MobileNet architecture (Howard et al., 2017).

The pre-trained EfficientNetV2-B2 is available from the TensorFlow hub. This version of the EfficientNetV2-B2 has been trained on the ImageNet 21 K dataset. It has 10.1 million parameters and the number of floating-point operations per second (FLOPs) is 1.7 Billion (TensorFlow Hub, 2023). The last layer of the EfficientNetV2-B2 is connected to a neural network with two dense layers of 512 and 11 neurons respectively. The first dense layer has an activation layer of ReLU and the second dense layer has a Softmax activation function.

3.3. Evaluation metrics

To evaluate the models from our experiments, we are focusing on the top-1 accuracy metric, which describes how the model performs across all the different classes. It is usually represented as a ratio of correct predictions over the total number of predictions. The correct predictions are represented as True Positives (TP) and True Negatives (TN), while the total number of predictions also include the False Positives (FP) and False Negatives (FN), as per Eq. (3). Additionally, since the dataset has some degree of imbalance for a few classes, Precision Eq. (4), Recall Eq. (5) and F1-Score Eq. (6) are also being used to provide a more realistic comparison.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{3}$$

$$Precision = \frac{TP}{TP + FP} \tag{4}$$

$$Recall = \frac{TP}{TP + FN} \tag{5}$$

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (6)$$

3.4. Experimental setup and configurations

All our experiments have been conducted on the following hardware architecture, including model training and testing:

- i7-3770k CPU
- 16 GB Ram
- GTX 1650 4 GB DDR5 GPU
- Window 10 Operating System

4. Results and discussion

4.1. Experimental results

Following different experiments, we present the results from our three models using the most relevant hyperparameter configurations starting with and without data augmentation. The batch size defines the number of samples from the dataset used for each training cycle, known as epoch. The larger the batch size, the more computation the models perform during the training phase. The minimum batch size used is 64 and the maximum is 256, and the metrics recorded for analysis are the accuracy, precision, recall and F1-score. Table 5 presents the measurement for the validation phase of the different models.

It can be observed from Table 5, that firstly data augmentation contributes to increasing the validation accuracy, precision and recall for each of the models. Secondly, the transfer learning models based on InceptionV3 and EfficientNetV2-B2 have better results than the model built from scratch. Thirdly, a larger batch size used during training has a positive impact on the results. Further experiments, with different hyperparameter tuning for Model 1, the CNN built from scratch, will prove to be time consuming and inefficient for food type classification, given the large variety of images for the same type of food. Transfer learning, as can be seen from Model 2 and 3, has better prediction capabilities. Comparing the transfer learning models using data augmentation, it is noticed that the EfficientNetV2-B2 has the highest accuracy of 94.5 %. This significant increase is mainly due to the model's parameters which has been generated by learning the features from millions of images of the ImageNet dataset. Also, EfficientNetV2-B2 used a unique combination of Fused-MBConv and MBConv layers, along with standard convolution, pooling and fully connected layers, contributing to its improved efficiency.

Model 1 and Model 2 performed poorly without data augmentation, with a relatively low precision and recall values, implying that they have higher probability of predicting false positives and negatives. Model 3 has proved to be quite efficient especially with increased batch sizes during training. Without data augmentation and with batch size of 256, Model 3 achieves an accuracy of 94.02 % and F1-score of 93.85 %. With data augmentation, the accuracy increased to 94.52 % and F1-score to 94.67 %. It implies that this model can correctly classifies positive cases, with a small margin of error.

Our experiments were configured to use early stopping, to avoid degradation of the performance during validation, and thus avoiding the problem of overfitting. Therefore, as can be seen from Table 6, the batch size corresponding to the number of images to be generated for each step of an epoch has been increasing for each experiment. The number of epochs for each model differs, as well as the number of steps in each epoch. Setting a fixed value creates a trade-off with early stopping, thus increasing the risk of the model overfitting. On average, around 15,884 images have been generated on top of the initial 16,643 from the dataset in each epoch. Combining the results presented in Table 5 and 6, we note that an increased batch size contributes to improving the model's feature identification capabilities, as more images with different augmentations are being used during the training phase. However, this is restricted by the amount of memory available on the machine, limiting our experiments to a batch size of 256.

4.2. Comparative analysis

For further assessment, we compared the results of our experiments with similar studies which have some degree of consideration for avoiding overfitting. The results are presented in Table 7. Since only a few studies used the same Food-11 dataset, we also included studies based on the Food-101 dataset.

Our study outperforms all other approaches which used the Food-11 dataset, with the highest increase in prediction accuracy being 11 % while the minimum increase is around 5.2 %. Although some approaches used transfer learning, none of them used the EfficientNetV2 as

Table 6
Number of images generated randomly with data augmentation.

Experiment No.	CNN Models – With Data Augmentation	Batch Size	Steps per Epoch	Epochs	No. Images Generated per Epoch
2	Model 1 - From Scratch (with data augmentation)	64	309	39	19776
4	Model 2 - InceptionV3 (with data augmentation)	64	155	14	9920
8	Model 3 – EfficientNetV2-B2 (with data augmentation)	64	309	24	19776
9	Model 3 – EfficientNetV2-B2 (with data augmentation)	128	78	34	9984
10	Model 3 – EfficientNetV2-B2 (with data augmentation)	256	78	28	19968
Average number of images generated per experiment within each Epoch					15884

Table 5
Experimental results.

Experiment No.	CNN Models	Batch Size	Accuracy	Precision	Recall	F1-Score
1	Model 1 – From Scratch (without data augmentation)	64	0.4930	0.5317	0.4940	0.4942
2	Model 1 - From Scratch (with data augmentation)	64	0.6464	0.6982	0.6453	0.6336
3	Model 2 - InceptionV3 (without data augmentation)	64	0.8569	0.8878	0.8564	0.8572
4	Model 2 - InceptionV3 (with data augmentation)	64	0.8327	0.8646	0.8339	0.8322
5	Model 3 – EfficientNetV2-B2 (without data augmentation)	64	0.9137	0.9563	0.9442	0.9437
6	Model 3 – EfficientNetV2-B2 (without data augmentation)	128	0.9362	0.9424	0.9363	0.9360
7	Model 3 – EfficientNetV2-B2 (without data augmentation)	256	0.9402	0.9420	0.9386	0.9385
8	Model 3 – EfficientNetV2-B2 (with data augmentation)	64	0.9315	0.9411	0.9320	0.9295
9	Model 3 – EfficientNetV2-B2 (with data augmentation)	128	0.9423	0.9475	0.9421	0.9421
10	Model 3 – EfficientNetV2-B2 (with data augmentation)	256	0.9452	0.9481	0.9454	0.9467

Table 7
Comparative analysis with existing studies.

Reference	Model	Dataset	Accuracy (%)
Singla et al. (2016)	GoogLeNet	Food-11	83.5
Islam et al. (2018a)	RestNet-50	Food-11	88.09
Özsert Yiğit et al. (2018)	AlexNet	Food-11	86.9
Sengur et al. (2019)	AlexNet & VGG16 + SVM for classification	Food −11	89.3
This Study	**EfficientNetV2-B2**	**Food-11**	**94.5**
Tan and Le (2021a, 2021b)	EfficientNet-B7	Food-101	93.0
Touvron et al. (2020)	Grafit (RegNet-8GF)	Food-101	93.7
Ghalib & Chu (2021)	MobiletNetV3	Food-101	80.8
Vijayakumari et al. (2022)	EfficientNet-B0	Food-101	80.0

base model. EfficientNetV2 reduces training time, while making more efficient use of the parameters, using a progressive learning approach. This model also improves on EfficientNetV1 by using smaller images for training and avoiding slow depth-wise convolutions. In the first convolution layers of MBCov, the conv 1x1 and conv 3x3 from version 1 have been replaced with a standard 3x3 convolution, known as Fused-MBConv (Tan & Le, 2021a, 2021b). Although this work used the Food-11 dataset, we also compared our results with different approaches which used the Food-101 dataset. It can be observed our approach based on the EfficientNetV2 has a higher accuracy value.

As compared to different other studies, our approach is not overfitting. It has the potential to generalise well on unseen data. It can be noted from Fig. 7, that the training and validation accuracy, after 8 epochs, follow a rather similar path without diverging. Also, it can be seen that the model has not learned all the different intricacies in the images, with the maximum training accuracy being below 95 %. The use of data augmentation, drop out and early stopping have contributed to keep the model safe from overfitting.

5. Conclusion and future works

In this study, we applied different deep learning techniques to identify and classify different types of food from images. In the first approach we built and configured a Convolutional Neural Network from scratch. It did not perform very well with a prediction accuracy of only 64.6 %, and required a tedious parameter configuration process. The second approach was based on transfer learning and achieved a much better accuracy of 94.5 %. Using an optimised model, pre-trained on a very large dataset with various types of images effectively contributed to the configuration of parameters needed for food type prediction. This task remains quite challenging for computer vision, as dishes with a minor change in ingredients or presentation style can sometimes lure the human eyes to incorrectly identifying them. However, with minimal configurations, using transfer of knowledge from a different context, our pre-trained model achieved a significant improvement in the detection of 11 categories of food. With an F1-score of 94.7 %, the model can correctly classify positive cases, with a very small margin of error. Also, our approach successfully avoids overfitting through data augmentation, drop out and early stopping techniques. Hence the model generalizes with a low variance.

This work demonstrates the potential for an efficient automatic recommender system, perhaps in the form of a mobile application, which uses the camera. The appropriate type of food based on a specific diet or different health conditions can be proposed with a higher degree of confidence. The main limitation remains the detection and recognition of different ingredients in a dish, which is mainly due to the unavailability of large varieties of food image datasets with detailed labels.

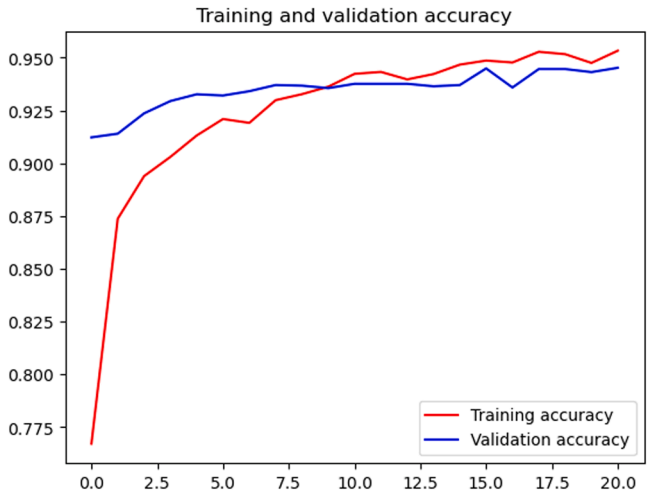

Fig 7. Training and validation accuracy (y-axis) against epochs (x-axis) for model 3 – EfficientNetV2-B2 (with data augmentation).

Listing the ingredients in each food image gathered may assist deep learning models to perform better in the food classification challenge. Future works will focus on the assessment of how models determine features in food images using the Gradient-weighted Class Activation Mapping (Grad-CAM) technique.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Attokaren, D.J., Fernandes, I.G., Sriram, A., Murthy, Y.V. S., & Koolagudi, S.G., (2017) "Food classification from images using convolutional neural networks," TENCON 2017 - 2017 IEEE Region 10 Conference, Penang, Malaysia, 2017, pp. 2801–2806, doi: 10.1109/TENCON.2017.8228338.

Bengio, Y., Goodfellow, I. & Courville, A., 2017. Deep learning, Massachusetts: MIT Press.

Bossard, L., Guillaumin, M., Van Gool, L. (2014). Food-101 – mining discriminative components with random forests. In: Fleet, D., Pajdla, T., Schiele, B., Tuytelaars, T. (eds) Computer vision – ECCV 2014. ECCV 2014. Lecture Notes in Computer Science, vol 8694. Springer, Cham. https://doi.org/10.1007/978–3-319–10599-4_29.

Ciocca, G., Paolo, N..,Raimondo, S., (2017). Learning CNN-based Features for Retrieval of Food Images. Battiato, Sebastiano and Farinella, Giovanni Maria and Leo, Marco and Gallo, Giovanni (Eds) New Trends in Image Analysis and Processing – ICIAP 2017: ICIAP International Workshops, WBICV, SSPandBE, 3AS, RGBD, NIVAR, IWBAAS, and MADiMa 2017, Catania, Italy, September 11–15, 2017, Revised Selected Papers", Springer International Publishing, 426—434, 978–3-319–70742-6, doi="10.1007/978–3-319–70742-6_41.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009) ImageNet: A Large-Scale Hierarchical Image Database. IEEE Computer Vision and Pattern Recognition (CVPR).

Dhanya, V. G., Subeesh, A., Kushwaha, N. L., Kumar Vishwakarma, Dinesh, Kumar, Nagesh, Ritika, T., & Singh A.N, G. (2022). Deep learning based computer vision approaches for smart agricultural applications. ISSN 2589-7217 *Artificial Intelligence in Agriculture* (Volume 6,(2022), 211–229. https://doi.org/10.1016/j.aiia.2022.09.007.

Everingham, M., Eslami, S. M. A., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2015). The PASCAL visual object classes challenge: A Retrospective. *International Journal of Computer Vision, 111*(1), 98–136.

Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. In Proceedings of International Conference on Learning Representations.

Ghalib, A. T., & Chu, K. L. (2021). Explainable deep learning ensemble for food image analysis on edge devices. ISSN 0010-4825 *Computers in Biology and Medicine* (Volume 139)(2021), Article 104972. https://doi.org/10.1016/j.compbiomed.2021.104972.

Griffin, G., Holub, A., & Perona, P. (2022). Caltech 256 (1.0) [Data set]. CaltechDATA. https://doi.org/10.22002/D1.20087.

He, K.; Zhang, X.; Ren, S.; Sun, J. (2016). "Deep residual learning for image recognition". In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.

Howard, A.G., Zhu, M.L., Chen, B., Kalenichenko, D., Wang, W.J., Weyand, T., Andreetto, M. and Adam, H. (2017) MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.

Islam, K. T., Wijewickrema, S., Pervez, M., & O'Leary, S. (2018b). An exploration of deep transfer learning for food image classification, 2018 *Digital Image Computing: Techniques and Applications (DICTA)*. https://doi.org/10.1109/dicta.2018.8615812.

Islam, M.T., Karim Siddique, B.M. N., Rahman, S., & Jabid, T. (2018a). Food Image Classification with Convolutional Neural Network. 2018 International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS). doi:10.1109/iciibms.2018.8550005.

Hokuto Kagaya, Kiyoharu Aizawa, and Makoto Ogawa (2014). Food detection and recognition using convolutional neural network. In Proceedings of the 22Nd ACM International Conference on Multimedia, MM '14, pages 1085–1088, New York, NY, USA, 2014. ACM.

Kawano, Y. and Yanai, K. (2014) Automatic Expansion of a Food Image Dataset Leveraging Existing Categories with Domain Adaptation, In proceedings of ECCV Workshop on Transferring and Adapting Source Knowledge in Computer Vision (TASK-CV)", 2014.

Kim, B., Yuvaraj, N., Park, H. E., Preethaa, K. R. S., Pandian, R. A., & Lee, D. (2021). Investigation of steel frame damage based on computer vision and deep learning. *Automation in Construction* (Vol. 132)(2021), Article 103941. ISSN 0926-5805.

Kiourt, C., Pavlidis, G., & Markantonatou, S. (2020). *Deep learning approaches in food recognition, Machine Learning Paradigms- Advances in Theory and Applications of Deep Learning*. Springer.

Krizhevsky, Alex, Ilya, Sutskever, & Geoffrey, Hinton (2012). *ImageNet Classification with Deep Convolutional Neural Networks Neural Information Processing Systems*, 25. https://doi.org/10.1145/3065386

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278–2324. https://doi.org/10.1109/5.726791

Lee, K., He, X., Lei, Z.& Linjun, Y.. (2018). CleanNet: Transfer Learning for Scalable Image Classifier Training with Label Noise. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR.

Lin, T., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan D., Zitnick, C.L., & Dollar, P. (2015). Microsoft COCO: Common Objects in Context.

Lopes, J.F., da Costa, V.G.T., Barbin, D.F., Cruz-Tirado L.J.P., Baeten V.& Barbon Junior S. (2022) Deep computer vision system for cocoa classification. Multimedia Tools Applications, 81, 41059–41077 (2022). https://doi.org/10.1007/s11042–022-13097–3.

Ma, P., Lau, C. P., Yu, N., Li, A., & Sheng, J. (2022). Application of deep learning for image-based Chinese market food nutrients estimation. *Food Chemistry, 373* (130994), 2022. https://doi.org/10.1016/j.foodchem.2021.130994

Matsuda, Y. and Hoashi, H. and Yanai, K. (2012). Recognition of Multiple-Food Images by Detecting Candidate Regions, In Proceedings of IEEE International Conference on Multimedia and Expo (ICME)", 2012.

Oliveira, M. M., Cerqueira, B. V., Barbon, S., & Barbin, D. F. (2021). Classification of fermented cocoa beans (cut test) using computer vision. ISSN 0889-1575 *Journal of Food Composition and Analysis* (Volume 97)(2021), Article 103771. https://doi.org/10.1016/j.jfca.2020.103771.

Özsert Yiğit, G., & Özyildirim, B. M. (2018). Comparison of convolutional neural network models for food image classification. *Journal of Information and Telecommunication, 2* (3), 347–357. https://doi.org/10.1080/24751839.2018.1446236

Qian, L., Hu, L., Zhao, L., Wang, T., & Jiang, R. (2020). Sequence-Dropout block for reducing overfitting problem in image classification. *IEEE Access, 8*, 62830–62840. https://doi.org/10.1109/access.2020.2983774

Qian, L., Hu, L., Zhao, L., Wang, T. and Jiang, R. (2020b) "Sequence-Dropout Block for Reducing Overfitting Problem in Image Classification," in IEEE Access, vol. 8, pp. 62830–62840, 2020, doi: 10.1109/ACCESS.2020.2983774.

Russakovsky, Olga, Deng, Jia, Su, Hao, Krause, Jonathan, Satheesh, Sanjeev, Ma, Sean, Huang, Zhiheng, Karpathy, Andrej, Khosla, Aditya, Bernstein, Michael, Alexander, C. Berg, & Fei-Fei, Li (2015). ImageNet large scale visual recognition challenge. *International Journal of Computer Vision IJCV, 2015*.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2020). Grad-CAM: visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision, 128*(2), 336–359, 2020.

Şengür, A., Akbulut, Y., & Budak, Ü. (2019). Food image classification with deep features. *In 2019 International Artificial Intelligence and data Processing Symposium (IDAP)*, 1–6.

Shao, W., Min, W., Hou, S., Luo, M., Li, T., Zheng, Y., & Jiang, S. (2023). Vision-based food nutrition estimation via RGB-D fusion network. ISSN 0308-8146 *Food Chemistry* (Volume 424,(136309), 2023. https://doi.org/10.1016/j.foodchem.2023.136309.

Simonyan, k., and Zisserman, A. (2015), Very Deep Convolutional Networks for Large-Scale Image Recognition, International Conference on Learning Representations, 2015.

Singla, Ashutosh., Yuan, Lin. & Ebrahimi, Touradj (2016). Food/Non-food Image Classification and Food Categorization using Pre-Trained GoogLeNet Model. In Proceedings of the 2nd International Workshop on Multimedia Assisted Dietary Management, 2016, Pages 3–11, ACM. https://doi.org/10.1145/2986035.2986039.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016) Rethinking the Inception Architecture for Computer Vision. In Proceedings of 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 2818–2826, doi: 10.1109/CVPR.2016.308.

Tan, M.& Le, Q.V. (2021a). EfficientNet: Rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning, Long Beach, California.

Tan, M.& Le, Q.V. (2021b). EfficientNetV2: Smaller Models and Faster Training. Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021.

TensorFlow Hub, Retrieved 24 March 2023 from:⟨<underline>https://tfhub.dev/google/imagenet/efficientnet_v2_imagenet21k_b2/feature_vector/2</underline>⟩.

Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A.& Jégou, H.. (2020). Training data-efficient image transformers & distillation through attention. Proceedings of the 38th International Conference on Machine Learning, PMLR 139.

U.S. Department of Health and Human Services and U.S. Department of Agriculture (2015). 2015 – 2020 Dietary Guidelines for Americans. 8th Edition. December 2015. Retrieved 28 March 2023 from: ⟨<underline>https://health.gov/our-work/food-nutrition/previous-dietary-guidelines/2015</underline>⟩.

Vijaya Kumari, G., Vutkur, Priyanka, & Vishwanath, P. (2022). Food classification using transfer learning technique, 2022 *Global Transitions Proceedings* (Vol. 3,(1), 225–229. https://doi.org/10.1016/j.gltp.2022.03.027.

World Health Organization, WHO (2020). Healthy Diet. Retrieved March 2023 from: https://www.who.int/news-room/fact-sheets/detail/healthy-diet.

Xiao, J., Hays, J., Ehinger, K., Oliva, A., and Torralba, A. (2010). "SUN Database: Large-scale Scene Recognition from Abbey to Zoo". IEEE Conference on Computer Vision and Pattern Recognition, 2010.

Xu, Q., Zhang, M., Gu, Z., & Pan, G. (2019). Overfitting remedy by sparsifying regularization on fully-connected layers of CNNs 2019. *Neurocomputing* (Vol. 328,, 69–74. https://doi.org/10.1016/j.neucom.2018.03.080

Xu, Qi, Zhang, Ming, Zonghua, Gu, & Pan, Gang (2019). Overfitting remedy by sparsifying regularization on fully-connected layers of CNNs. ISSN 0925-2312 *Neurocomputing* (Volume 328,, 69–74. https://doi.org/10.1016/j.neucom.2018.03.080.

Keiji Yanai and Yoshiyuki Kawano. Food image recognition using deep convolutional network with pre-training and _ne-tuning. In Multimedia & Expo Workshops (ICMEW), 2015 IEEE International Conference on, pages 1{6. IEEE, 2015.

Yoshiyuki Kawano and Keiji Yanai. Food image recognition with deep convolutional features. In Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous computing: Adjunct Publication, pages 589–593. ACM, 2014.

Zakour, J.M., Swager, M.C., (2018). Vulnerability-plus theory: The integration of community disaster vulnerability and resiliency theories. In M. J.Zakour, N. B. Mock, P. Kadetz (Eds.), Creating Katrina, Rebuilding Resilience (pp, 45–78). Butterworth-Heinemann.

Zhu, Z., & Dai, Y. (2021). Food ingredients identification from dish images by deep learning. *Journal of Computer and Communications, 9*, 85–101. https://doi.org/10.4236/jcc.2021.94006